HTs-GCN: Identifying Hardware Trojan Nodes in Integrated Circuits Using a Graph Convolutional Network

Jie Xiao, *Member, IEEE*, Shuiliang Chai, *Graduate Student Member, IEEE*, Yanjiao Gao, Yuhao Huang, Fan Zhang, *Member, IEEE*, and Tieming Chen

***Abstract*—Hardware Trojans (HTs) present significant security threats to integrated circuits. Detecting and locating HTs is crucial for mitigating these threats. Thus, this article proposes a method called HTs-GCN, which utilizes a graph convolutional network (GCN) to identify HTs. First, it extracts two novel features of gate nodes using a depth-first search strategy and topological logical analysis to enrich the feature information of circuit nodes. Second, through a message-passing mechanism, it designs a local feature aggregation method based on the GCN and a global feature fusion method based on an attention mechanism to improve the representation capability of circuit node features. Then, leveraging the concept of stochastic gradient descent and incorporating mini-batch oversampling and undersampling techniques, it employs a dataset imbalance handling method to address the scarcity of HT nodes in circuits. These approaches significantly enhance the distinguishability between gate nodes with HTs and other gate nodes while reducing computational complexity. Experimental results indicate that HTs-GCN outperforms the recently proposed NHTD-GL method in terms of recall: it achieves approximately 7.8% points higher recall while maintaining similar accuracy. HTs-GCN demonstrates exceptional generalizability, with an average recall and accuracy of 93.0% and 100%, respectively, on infrequently used circuits in the Trust-Hub benchmark. In addition, on the TRIT-TC benchmark, HTs-GCN achieves excellent average true positive rate (TPR) and true negative rate (TNR) of 95.1% and 94.4%, respectively. Furthermore, HTs-GCN exhibits robust performance under gate modification attacks, with average TPR and TNR reaching 82.1% and 92.5%, respectively.**

***Index Terms*—Data balancing, feature fusion, hardware Trojans (HTs), node features.**

I. Introduction

With the growing demand for high-performance, low-cost, and multifunctional integrated circuits (ICs), enterprizes and designers are increasingly relying on the incorporation of third-party intellectual property (3PIP) cores or outsourcing standard and universal logic designs to specialized design firms to gain a competitive edge. This strategy enables them to concentrate on the development of new functionalities; however, it simultaneously increases the risk of chips being subjected to malicious hardware attacks. For example, untrusted 3PIP suppliers or design contractors might embed hardware Trojans (HTs) in their designs [1]. HTs are lightweight components within large-scale complex ICs, consisting of Trojan triggers and payloads. Most HTs are activated only under specific circumstances, making them difficult to detect using conventional verification and testing methodologies. Once the triggers of HTs are activated, they can cause confidential information stored in the IC to leak or be tampered with and even lead to IC chip failure or damage. Consequently, it is essential to develop efficient methodologies for the rapid detection of HTs.

In recent years, the industry has proposed a series of HT detection methods [2], [3], [4], [5], [6] that rely on structural features rather than simulation, providing an in-depth and comprehensive analysis of the target IC design. Compared with traditional logic testing methods [7], HT detection methods based on structural and functional characteristics usually have a more efficient capability for comprehensive analysis of circuit node features. For example, without threshold limitations, Kok et al. [2] employed a bagged tree classification model based on integrated learning to effectively detect HTs by manually extracting Sandia controllability and observability analysis program (SCOAP) values manually from the benchmark circuits [8]. SCOAP assesses circuit testability by measuring the difficulty of forcing each node to output a specific logic 0 or 1; it also measures the difficulty of observing that specific logic at the primary outputs. Huang and He [9] demonstrated that the difficulty of forcing a node to output a specific logic 0 or 1 does not always correlate positively with the probability of it outputting that specific logic. This implies that the difficulty of forcing a node to output a specific logic does not effectively measure its stealthiness. The method presented in [3] achieved high

Received 28 March 2024; revised 30 July 2024 and 6 November 2024; accepted 14 December 2024. Date of publication 19 December 2024; date of current version 21 May 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3106800; in part by the Natural Science Foundation of Zhejiang Province under Grant LR24F020003 and Grant LZ22F020011; and in part by the National Natural Science Foundation of China under Grant 62472386 and Grant 62072398. This article was recommended by Associate Editor A. Davoodi. *(Corresponding author: Fan Zhang.)*

Jie Xiao is with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310000, China, and also with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China (e-mail: xiaojiexqj@foxmail.com).

Shuiliang Chai, Yanjiao Gao, Yuhao Huang, and Tieming Chen are with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310000, China (e-mail: tmchen@zjut.edu.cn).

Fan Zhang is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310058, China (e-mail: fanzhang@zju.edu.cn).

Digital Object Identifier 10.1109/TCAD.2024.3520522

true positive and true negative rates (TNRS) for HT detection by utilizing a random forest classifier based on 36 structural features reflecting HT trigger circuits extracted manually.

However, manual feature extraction heavily depends on the experience and knowledge of design engineers. If the extracted IC features are of low quality, the model's detection accuracy for HTs will be significantly diminished. To address these shortcomings, researchers have proposed various machine learning (ML)-based HT detection methods [10], [11]. For example, Yasaei et al. [11] constructed a multilayer perceptron (MLP)-based HT detection method by utilizing support vector machines and artificial neural networks to learn HT features. This method achieved a higher recall, but its accuracy was insufficient. Recently, several ML-based HT detection methods have been proposed boasting 90% or even higher accuracies [12], [13].

However, traditional ML-based HT detection methods frequently encounter significant challenges in identifying appropriate feature sets: the first limitation is the need for deep data understanding and complex preprocessing, which renders them unsuitable for large-scale datasets. For instance, Ghandali et al. [14] and Zhang et al. [15] detected HTs in circuits based on extracted features such as temperature and delay. However, extracting temperature features necessitates the use of high-precision sensors along with calibration and noise reduction techniques to guarantee data accuracy and stability. Meanwhile, extracting delay features depends on high-resolution timing analysis tools and requires clock synchronization to ensure measurement accuracy. These intricate preprocessing steps present substantial challenges when applying such methods to large-scale datasets. The second limitation is that the extraction of some features requires extensive computational resources and time, which is often unsustainable in practice. For example, the HT detection and localization method proposed in [4] often suffered from significant time overhead due to its reliance on the simulation-based computation of logic gate flipping probabilities. In fact, the simulation often involves setting some boundary conditions, which increases computation complexity [16]. And the third limitation is that numerous studies predominantly adopt heuristic approaches in the quest to detect HTs, thereby relying on experiential knowledge,intuitive reasoning, and prevailing beliefs instead of adhering to rigorous mathematical formulations or systematic analytical procedures. This predilection for heuristics, while potentially yielding rapid and efficient solutions, renders these methods susceptible to circumvention by sophisticated adversaries. For example, in the selection of features such as the number of fan-ins at levels 1–5 from a node input, Kurihara and Togawa [3] and Hasegawa et al. [6] relied more on empirical methods rather than strict mathematical derivation, which resulted in varying HT detection performance across different types of circuits.

To overcome the aforementioned limitations of traditional ML-based methods, researchers have introduced graph learning (GL) methods to detect HTs in ICs [17]. Graph neural networks (GNNs) can exploit their inherent strengths to deeply explore the characteristics of each basic gate and the overall design structure to predict HTs. This led to GNN-based HT detection methods receiving widespread attention and emphasis from the industry [18], [19]. For instance, Lashen et al. [5] utilized a sampling-based method to extract smaller subgraphs from larger original graphs for training. By implementing a threshold selection strategy, the constructed GNN model was able to focus more on the minority class, thereby improving the identification of HTs. However, this threshold selection and the application of sampling strategies introduced scalability challenges. Drawing on "general knowledge" about known circuits, Pan and Mishra [10] trained a graph convolutional network (GCN) to memorize the characteristics and underlying attributes of normal and HT circuits. Their GCN then leveraged this prestored "general knowledge" to differentiate between circuits with and without HTs based on similarity scores.

Despite this progress, three limitations remain major obstacles to the wider application of existing GNN-based HT detection methods. First, there is the localization issue of HTs. Although recently proposed GNN-based the most advanced HTs detection schemes based on GNN, such as GNN4TJ [18], HW2VEC [19], and Td-zero [10], can detect whether an IC design contains HTs, they cannot be used to locate the HTs within it. Extensive research has demonstrated that precise localization of HTs is advantageous for identifying defective sections in IC designs and ascertaining the accountable parties. Second, there is the issue of stealth. The covert nature of HTs means they are difficult to trigger and activate under normal operating conditions. For example, for a circuit with 150 rare nodes, if four of them are used for triggering, the defender needs to check as many as ${}^{150}C_4 \approx 20 \times 10^6$ different combinations of rare nodes [20]. This makes HTs in IC designs hard to detect using traditional verification and test methods [6], and GNN models may also face challenges when dealing with this stealth issue [4]. Moreover, there is an imbalance issue in the datasets. HTs are small, and their gate count constitutes only a tiny fraction of the overall IC design. Taking the Trust-Hub benchmark as an example, the gate count of HTs constitutes a mere 0.13-9.29% of the overall design, with an average of approximately 1.73%. This can result in performance degradation for the ML models when dealing with similar imbalanced datasets. In practice, this issue is a relatively common challenge in ML, and it often impacts the generalizability and accuracy of the applied models [5].

To overcome the limitations of existing methods, this article proposes a method named HTs-GCN for HT detection and localization leveraging both structural and functional features alongside a GCN model. First, GL methods are employed to convert circuit structures into graph structures and extract fundamental structural features of the circuits upon completion of the transformation. These features have been proven by existing works [4] to be effective in distinguishing HTs from normal nodes. Second, two crucial new features, Cov and logical flip probability (Lfp), are constructed using a deep search strategy and topological logic analysis, respectively. These new features, combined with the foundational structural characteristics, generate the initial node-level feature vectors. Cov can effectively reflect the depth and breadth of the adverse impact of HTs on a circuit, while Lfp can better expose HTs with higher concealment in a circuit. Combined with the basic structural features of a circuit, they can be used

effectively to overcome the problems of strong concealment and difficult localization of HTs. Subsequently, using the initial node feature vectors as input, a GCN is employed to fuse the local information of a node through an aggregation strategy. Afterward, the final node feature vector is obtained by fusing the global feature vector derived from an attention mechanism with the aggregated node feature vector. This feature fusion strategy further amplifies the expressive ability of each node's features within the circuit, thereby improving the ability to address the challenges of high concealment and difficult localization of HTs. In addition, a method for handling dataset imbalance is developed based on stochastic gradient descent and incorporating mini-batch oversampling and under-sampling techniques. This approach addresses the dataset imbalance caused by the scarcity of HT nodes in a circuit, thereby enhancing HTs-GCN's generalizability and accuracy. Finally, based on the balanced dataset, a well-trained MLP model is utilized to detect and locate different types of HTs in a circuit. The contributions of this article can be summarized as follows.

1) Two key features, Cov and Lfp, are constructed utilizing deep search strategies and topological logic analysis, respectively. Cov quantifies the connectivity of nodes in a graph, while Lfp approximately calculates the probability of logical value flipping at a node. These enrich the feature information of circuit nodes.
2) We design a local feature aggregation method based on GCN and a global feature fusion method based on the attention mechanism. This method involves using a message-passing mechanism to aggregate neighboring feature information of circuit nodes and to fuse the global feature information of the entire circuit. The resulting node feature vectors act as sample data for the final classification model, thus effectively leveraging both local and global circuit node information. This enhances the feature representation capabilities of the circuit nodes and enables the classification model to better capture intrinsic data relationships and improve performance on complex tasks.
3) We construct a method to address dataset imbalance by leveraging stochastic gradient descent, combined with mini-batch oversampling and under-sampling techniques. In this way, we can resolve the issue of HT node scarcity in the dataset. Our oversampling strategy generates HT nodes in each mini-batch to balance the dataset by sampling from the HT set, while the mini-batch data generated by the under-sampling strategy is used to improve the generalizability and training speed of the model.

The experiments were conducted on the Trust-Hub and TRIT-TC benchmarks using the industry's commonly employed leave-one-out cross-validation strategy. The experimental results indicate that on the Trust-Hub benchmark circuit, HTs-GCN achieved an average recall, average precision, average F1-score, average accuracy, and average TNR of 96.8%, 94.5%, 95.4%, 99.8%, and 99.9%, respectively. In comparison to the recently proposed NHTD-GL method [4], although HTs-GCN is about 3.3% points lower in precision and has similar performance in terms of accuracy and TNR, it has significant advantages in recall and F1-score, which are about 7.8% points and 3.3% points better on average, respectively. Compared with the methods in [2] and [3], the proposed method has an absolute advantage in all other metrics except for precision and TNR, which are similar to those of the method in [3]. Compared with TrojanSAINT [5], although HTs-GCN is about 1.5% points lower in recall, it is about 3.7% points better in TNR. Furthermore, HTs-GCN can be applicable to larger scale circuits and different dataset circuits. On the TRIT-TC benchmark circuits, HTs-GCN also has a true positive rate (TPR) of 95.1% and a TNR of 94.4%, which are about 0.2% and 9.1% points better than R-HTDetector [6] respectively. Under gate modification attacks, HTs-GCN outperforms R-HTDetector [6] by approximately 2.3% points and 7.1% points in TPR and TNR, respectively.

II. Background

A. Traditional HT Detection

Traditional HT detection methods mainly employ logic-based testing techniques [7] and side-channel analysis strategies [21]. Logic-based testing techniques focus on using logical properties to detect potential HTs. They typically generate numerous test patterns to activate potential HT circuits, resulting in observable errors or abnormal behavior that facilitate detection. For example, Mondal et al. [22] proposed an HT detection test pattern generation method based on the Monte Carlo (MC) method [23]. This approach involves generating test vectors either randomly or with constraints to perform sampling until the standard deviation of the measured signals across all samples falls within a specified accuracy range, thereby achieving HT detection. By combining action masking and exploration boosting strategies, Gohil et al. [20] constructed a reinforcement learning agent that efficiently generates a minimal test case set aimed at high HT coverage within a smaller search space. Based on the random insertion of 100 HTs into golden circuits such as ISCAS 85 and 89, the advancement of the agent is verified by comparing the size of the minimal test case sets generated by different methods and their coverage of HTs. Gohil et al. [20] focused on minimizing the test case set while maintaining high HT coverage, which differs from the objective of the present paper (i.e., efficiently detecting and locating all highly stealthy HTs in circuits). Therefore, the experimental design and evaluation metrics used in [20] significantly differ from those in the present paper. For instance, Gohil et al. [20] conducted experiments based on golden circuits, which do not require consideration of the high stealthiness of HTs, the differences in HTs triggering mechanisms, and robustness against adversarial attacks. The advantage of logic-based testing techniques is their ability to capture HTs at the logical level. However, they tend to be time-consuming on large-scale circuits due to their requirement for extensive test vector support. Side-channel analysis strategies often employ signal characteristics based on power, delay, and temperature analysis to detect changes in ICs and locate HTs. For example, Ghandali et al. [14] used side-channel analysis techniques to extract power consumption changes during AES

encryption kernel operation and applied a regression model to classify the extracted information for detecting HTs. This type of method conducts analysis by examining the physical characteristics generated by the circuits during runtime, such as power consumption and electromagnetic radiation [15]. Therefore, it can bypass logical complexity to detect some advanced HTs that are difficult to discover. However, due to the need for appropriate hardware devices and measurement techniques, these methods may be susceptible to interference from environmental noise, which leads to challenges in calibration and alignment, and thus resulting in some application limitations.

B. GL-Based HT Detection

In circuit design, netlists can naturally be transformed into graph structures where circuit components and connecting wires are represented as nodes and edges of the graph. This led to the emergence of GL-based HT detection methods as viable and innovative approaches in recent years. However, existing GL-based HT detection methods still possess limitations, such as some techniques only determining whether a design is affected by HTs without precisely locating the specific position of the HTs [11], or the method proposed in [13] focusing solely on the trigger part of the HTs and overlooking the crucial payload part. To overcome these limitations, Hasegawa et al. [4] integrated node detection techniques with circuits, GL, and HTs domain knowledge, and then they leveraged GL to capture potential features of HTs and employed a GNN for HT detection. This approach not only establishes a theoretical connection between GL and HT detection but also bridges the gap between the known structural characteristics of HTs and the representation capabilities of GNN, significantly improving the detection efficacy of HTs. However, when extracting certain initial node features such as the probability of a node outputting 0 or 1, the method often produces considerable time overhead due to requiring large simulations. Moreover, it focuses solely on local node features, which can lead to performance degradation when dealing with complex network structures. Although GL-based HT detection methods still have shortcomings, they offer a fresh perspective to address some of the issues in HT detection. For example, they can automatically capture the intricate dependencies between circuit topology and components using GL. Furthermore, GL demonstrates high flexibility and robust generalization capabilities, showing promise in adapting to different circuit topologies and HT attack patterns of varying scales. This attribute has resulted in the extensive use of GL across multiple domains including fault detection in smart grids [24]. Consequently, the present paper opts to utilize GL for the detection and localization of HTs in circuits.

III. HTs-GCN Methodology

Fig. 1 illustrates the framework flow of the proposed method for detecting and localizing HTs in circuits. First, the novel features Cov and Lfp are constructed based on the deep search strategy and topological logic analysis methods to enrich the circuit node feature information. Then, using the constructed initial node feature vectors as GCN input, a local information aggregation strategy is employed to aggregate neighboring node feature information. Subsequently, the graph representation method is utilized to extract the circuit global feature vector of the circuit, which is then continuously optimized through an attention mechanism-based iterative algorithm. On this basis, the optimized global feature vector is fused with the node feature vector to update the node feature vectors. Finally, a mini-batch training strategy is utilized to balance the training dataset to accelerate model training and improve its performance. The details are presented in Sections III-A, III-B, and III-C.

A. Constructing Features Cov and Lfp

Challenge 1: HT nodes represent a small percentage of nodes in a circuit and are often deliberately concealed, which makes it difficult to effectively identify them using existing structural features like node in-degree and out-degree features. In addition, existing functional features such as node Lfp struggle to effectively balance the enhancement of HTs' distinguishability and low computational overhead.

Solution 1: To address these challenges, we first construct a feature called Cov using a depth-first search strategy to quantify the connectivity of nodes within a graph. This allows us to measure the depth and breadth of the influence of HTs within a circuit. Next, we develop the feature Lfp based on topological logic analysis to approximate the Lfp of nodes, thereby efficiently addressing the issue of high computational overhead. The construction of these two new features facilitates the efficient identification of highly stealthy HTs in circuits.

Existing research indicates that connectivity quantifies the connectedness between a particular node and the remaining nodes in a graph, which reflects the positional information of the node in the graph [25]. In fact, connectivity is also one of the crucial factors determining whether HTs can play their intended roles in a circuit. When HT nodes possess high connectivity, they tend to efficiently interact and exchange information with other nodes, thereby making it easier for HTs to propagate malicious information or gain access to confidential information within the circuit. Consequently, in this article, connectivity is regarded as an important feature of nodes. Pan et al. [26] measured the connectivity of a node n_i by calculating the number of shortest paths between any two nodes excluding n_i. However, computing the shortest paths undoubtedly incurs significant computational costs as the circuit scale increases. Considering that connectivity can be measured by analyzing the accessibility of a node to the remaining nodes in the graph, this article designates nodes that are path-connected with node n_i as nodes with connectivity to n_i. The connectivity of n_i, $\mathrm{Cov}(n_i)$, is quantified by counting all such nodes in the graph, as per (1). Here, $\sum n_{\mathrm{from}}$ represents the total number of nodes in a directed graph that can reach node n_i, while $\sum n_{\mathrm{to}}$ denotes the number of nodes reachable from node n_i. The sum of these two values represents the total number of nodes in the graph having connectivity with node n_i. Furthermore, a normalization strategy

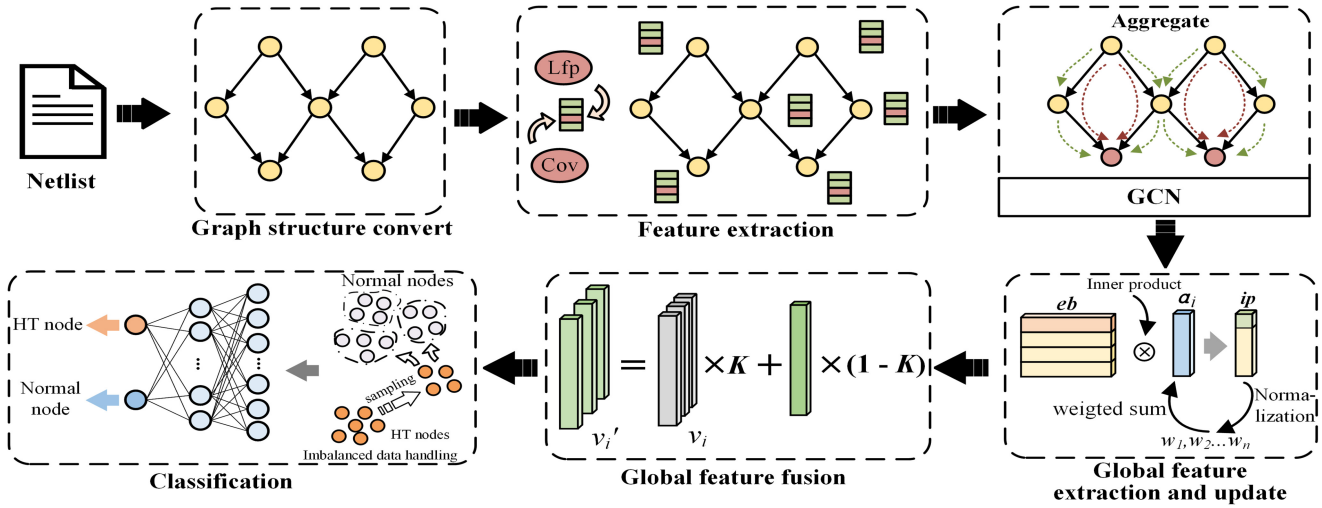

Fig. 1. Overview of HTs-GCN.

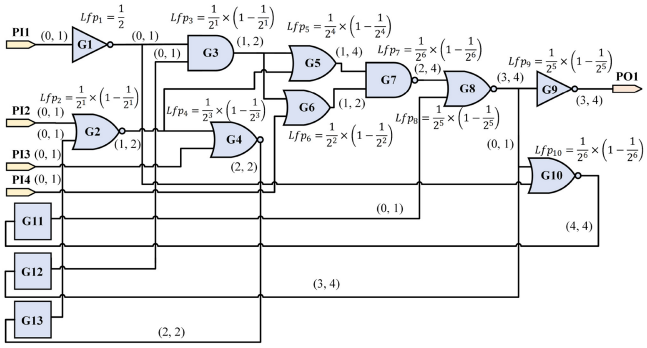

Fig. 2. Example of Lfp calculation on sequential circuit S27.

is adopted in this article to eliminate the potential negative impact of scale differences on the results

$$\mathrm{Cov}(n_i) = \frac{\sum n_{\mathrm{from}} + \sum n_{\mathrm{to}}}{n}, \quad 0 \le i \le n. \tag{1}$$

Taking node G3 in Fig. 2 as an example, G3 can be directly or indirectly connected to nodes G1, G5, G6, G7, G8, G9, G10, G11, and G12. Since the D-flip-flops (G11, G12, and G13) each contain five nodes internally, according to (1), the Cov value of G3 can be calculated as $17/25 = 0.68$.

In fact, attackers often exploit the rarity of nodes with low logic value flip probabilities in circuits. Inserting HTs into these types of nodes often helps to increase their concealment and destructiveness [4]. To this end, we introduce the Lfp to expose HTs with high concealment in a circuit. Two prevalent methods for calculating logic value flip probabilities for nodes are MC simulation [23] and equal-probability simulation [27]. The MC method leverages random sampling techniques to simulate node logic value flips for acquiring precise computational results. However, large-scale simulations often lead to substantial time costs. Assuming all input vectors have the same occurrence probability, the equal-probability simulation method [27] examines each node in a circuit layer by layer; this is guided by the serial and parallel structure characteristics among different nodes in the circuit. During this process, signal transmission simulations are expanded based on conditional probabilities to estimate the flip probabilities of logic values for each node. Compared with the MC method, this greatly reduces the number of samples required for simulation. However, due to the unsuccessful handling of the detrimental effects of fanout reconvergence on probability signals, computation accuracy is easily compromised, and the computational complexity increases exponentially with the scale of fanout branches. Existing research indicates that accurately calculating the logic value flip probability of circuit nodes is an NP-hard problem [28]. Therefore, according to actual application scenarios, developing lower-complexity approximation methods has become a common practice in the industry.

To obtain the probability of logic output values for circuit nodes, it is necessary to effectively quantify the direct or indirect influence of their parent nodes on these output values [29]. Analysis reveals that the output logic values of nodes are greatly influenced by the type of nodes along their input paths. Taking a node with a logic output value of 1 as an example, the probability of AND and NOR nodes along the input path also outputting 1 is relatively low. Conversely, other types of nodes along the input path exhibit a significantly higher probability of outputting 1. Similarly, when a node's logic output value is 0, the probability of OR and NAND nodes along the input path outputting 0 is relatively low; meanwhile, other node types present a significantly higher probability of outputting 0. To this end, based on information such as the number of input paths and the number of target nodes on the paths, this article proposes an approximate calculation method based on node influence attributes to estimate the Lfp of each node in a circuit. Since the probabilities of a node outputting 1 and 0 are complementary, to simplify calculations, we only focus on cases where the node output is 1. Furthermore, it is noted that the functions of all multi-input basic gates in a circuit can be achieved by series-parallel connections of multiple two-input basic gates, e.g., the AND-3 function can be implemented by two series-connected AND-2 gates. In addition, compared with other types of basic gates, AND and NOR gates have

the greatest impact on the probability of their child nodes outputting a logic value of 1. Hence, to reduce computational complexity, AND-2 and NOR-2 gates are considered target nodes in the node input paths.

Analysis has shown that for nodes with multiple input paths, the input path with the largest number of target nodes often has the greatest impact on the Lfp. Moreover, having multiple input paths with similar importance usually lowers a node's logic flip probability. For example, in Fig. 2, the two inputs of node G4 include three input paths originating from the primary inputs PI2, PI3, and D-flip-flop G13. Except for the path originating from PI3, the remaining two input paths have the same maximum target node count. Given a uniform input probability distribution, the probability of node G2 producing an output of 1 is 1/4, whereas the probability of PI3 generating an output of 1 is 1/2. From (2)–(4), it can be seen that the impact of G2 on the logic flip probability of G4 is significantly greater than that of PI3. If G4 has multiple input paths similar to G2, its logic flip probability will further decrease. Consequently, during the traversal of the circuit diagram, we not only extract the number of target nodes on each input path and the maximum target node count (*mt*) for each input port but also record the number of input paths containing the maximum target node count (*pn*) for each input port. Subsequently, a tuple (*mt*, *pn*) related to the target node information at the node input end is constructed based on the aforementioned information, and (2) is built through the principle of common causality [30] to quantify the degree of impact of relevant target nodes on graph nodes. Then, by integrating information on node attributes, such as type and in-degree, we quantify the probability of a node outputting 1. Equation (3) is a computational formula. When the node is a target node, with a dual-input basic gate as a baseline, the probability of the node outputting 1 decreases by half for each additional input port. For nontarget nodes, increasing the number of input ports does not significantly affect the probability of outputting 1. Therefore, this article does not consider the impact of such nodes on the final output. Finally, using the obtained probabilities of nodes outputting 1 and the method given in (4) in [27], we calculate the *Lfp* of nodes. It should be noted that we select target nodes to accelerate the calculation of Lfp for each node. As can be seen from (4), the calculation of node Lfp is based on the probability of the node outputting logic 1, which is complementary to the probability of its outputting logic 0. This complementarity is reflected in the fact that the target nodes corresponding to the case where the node outputs logic 0 are exactly the nontarget nodes in the case where the node outputs logic 1. Therefore, nontarget nodes are given the same degree of attention as target nodes in the calculation of Lfp in this article. For example, in Fig. 2, when the output of each node is 1, the target nodes corresponding to G7 are G2 and G3, and the probability of its logical output being 1 is $1/2^6$. When the output of each node is 0, the target nodes of G7 are G5 and G6, which is equivalent to the nontarget nodes when G7 outputs 1, and the probability of G7 outputting 0 is $(1-1/2^6)$. Lfp reflects the size of the logic flip probability of the nodes in a graph. Its calculation involves the number of corresponding target nodes and the number of associated paths for these nodes. Algorithm 1 outlines the specific calculation process of the Lfp of the circuit nodes, and the related calculation examples on the sequential circuit S27 can be seen in Fig. 2. Here, *nd* represents a certain node, *pi* signifies the set of primary input nodes, *rn* represents nodes in a graph excluding the primary input nodes, *nd.it* indicates the set of input ports of *nd*, and *pt* refers to the parent node of the current node

Algorithm 1 Lfp Calculating

Require: ***pi***, graph G
Ensure: Lfp
for *nd* in ***pi*** **do**
  Repeat $mt = 0, pn = 1$ **When** a in ***nd.it***;
  $ta(nd) = nd.idg - 1$;
  $po(nd) \leftarrow (3)$ and $Lfp(nd) \leftarrow (4)$;
end for
for *nd* in ***rn*** **do**
  for a in ***nd.it*** **do**
    if *a.pt* exist **then**
      if *a.pt* is *tn* **then**
        $mt = a.pt.mt.\max + 1$;
        $pn = \text{sum}(a.pt.pn \text{ if } a.pt.mt \text{ is max})$;
      else
        $mt = a.pt.mt.\max$;
        $pn = \text{sum}(a.pt.pn \text{ if } a.pt.mt \text{ is max})$;
      end if
    else
      $mt = 0, pn = 1$;
    end if
  end for
  $ta(nd) \leftarrow (2)$, $po(nd) \leftarrow (3)$ and $Lfp(nd) \leftarrow (4)$;
end for
return Lfp

$$ta_i = \sum_{k=1}^{n} \left(mt_k + pn_k - 1\right) \tag{2}$$

$$po_i = \begin{cases} \frac{1}{2^{ta_i}}, & i \neq t_n \\ \frac{1}{2^{ta_i + i.idg - 2}}, & i = t_n \end{cases} \tag{3}$$

$$Lfp_i = po_i \times \left(1 - po_i\right). \tag{4}$$

In (2), ta_i denotes the influence of the target node on the probability of node n_i outputting 1, and n represents the number of input ends for node n_i. pn_k - 1 quantifies the impact of multiple input paths with a maximum number of target nodes on node *Lfp*. In (3), po_i is the probability of node n_i outputting 1, *tn* signifies the type of target node, and *i.idg* indicates the in-degree of node n_i. In (4), Lfp_i represents the Lfp of node n_i.

In Algorithm 1, the *mt* and *pn* of each original input node in G are initialized based on the obtained graph structure G and according to the rules. Then, the *po* and *Lfp* of the corresponding nodes are initialized using (3) and (4). Next, the graph G is traversed layer-by-layer according to the breadth-first strategy, and (2)–(4) are used to quantify the ta, *po*, and *Lfp* for each node in G, excluding the primary input

nodes. Since each node and edge in G is traversed exactly once, with N denoting the number of nodes and E denoting the number of edges, the time overhead of this algorithm is approximately $O(N+E)$. The main objective of this algorithm is to approximate the calculation of the *Lfp* of nodes, which we use as an essential node feature in the HTs-GCN method. Because the required Lfp can be calculated with just one traversal of the graph structure, this algorithm exhibits low computational cost and is well-suitable for the analysis of complex large-scale circuits.

We obtained the *Lfp* of each node in Fig. 2 using Algorithm 1. Consider node G5 as an illustrative example. Algorithm 1 yielded the following $\langle mt, pn\rangle$ values for G5's two input ports: $\langle 1, 2\rangle$ and $\langle 1, 2\rangle$. Subsequently, we obtained the corresponding ta, *po*, and *Lfp* as 4, $1/2^4$, and $(1/2^4 \times (1 - 1/2^4))$ respectively. We verified that the Lfp of G5 obtained from Algorithm 1 is identical to the logic flip probability (15/256) obtained by the equiprobable simulation method [16]. Similar to the method in [17], we randomly generated the pseudo-primary inputs of the three D-flip-flops G11, G12, and G13 based on a uniform distribution.

In addition to the newly proposed node features Cov and Lfp, like in other methods [3], [4], basic structural features such as node gate type, in-degree, out-degree, shortest distance from the node to any primary input, and shortest distance from the node to any primary output, are also used for nodes. Table I presents all the node features utilized in the present work. Information such as the gate type, in-degree, and out-degree of a node reflects the different modes of signal propagation at the node [4]. These can all be extracted when converting the circuit structure into a graph structure using the GL method. There are a total of 40 types of gate nodes, which are represented by one-hot encoding. The shortest distance from each node to the primary input and output represents, respectively, the shortest distance from any primary input in the graph to that node and the shortest distance from that node to any primary output. These distances reflect the relative position information of a node in a graph [17], which helps us identify HTs with abnormal distances. For example, if the primary inputs are utilized as a subset of the HT trigger, the constructed local and global feature fusion strategy based on GCN and attention mechanisms can compensate for the absence of relative position information while highlighting other existing HT features.

Through this, we can enhance the node's feature information and aid the identification of such HTs. These distances can be calculated while traversing the circuit graph structure using breadth-first search. The aforementioned five basic structural features have all been demonstrated by existing research [4] to be effectively used to enhance the distinction between HTs and normal nodes. Cov quantifies the connectivity between any node in a graph and the remaining nodes, and it partially reveals the depth and breadth of an HT's influence throughout a circuit. Cov also encompasses features that reflect local connectivity, such as the number of neighbor nodes within multiple hops of a node [3], and it reflects the global connectivity of the node. Meanwhile, Lfp addresses the issue of excessive time overhead in calculating the logic flip probability of nodes in existing work [4]. It can effectively replace the functional features listed therein, such as the static logic value probability of nodes. Lfp not only helps to identify HTs with higher concealment in a circuit but also makes up for the controllability of SCOAP [9] and the specified logic probability of node output not always being positively correlated. For example, in Fig. 2, the controllability value SC^1 of node G4 specified to output logic 1 is 3, and the SC^1 of node G8 is 2; that is, the controllability of G4 is lower than that of G8. However, the probabilities of G4 and G8 outputting logic 1 are $1/2^3$ and $1/2^5$, respectively; this means that the probability of G4 outputting logic 1 is actually higher than that of G8. In summary, Cov and Lfp features can be utilized not only to distinguish abnormal HTs at the structural level but also to expose HTs with higher concealment at the functional level. Through their joint action, these features can effectively identify potential HTs in a circuit at multiple levels.

TABLE I
INITIAL FEATURE VECTOR USED IN THE PROPOSED MODEL

No.	Features
1–40	Node types
41	In-degree
42	Out-degree
43	Min. distance to any primary input
44	Min. distance to any primary output
45	Cov
46	Lfp

B. Fusing Local and Global Features

In node classification tasks, a GCN excels at aggregating information from neighboring nodes and therefore can handle large-scale graph data, learn complex structures, and improve local perception and generalization capabilities. Given that the proposed method only requires the integration of feature information from local neighboring nodes, a basic feature aggregation function of graph convolutional layers is sufficient for processing. In addition, considering the simplicity, efficiency, and suitability of the graph convolutional layer structure for handling graph data of various scales [4], it is employed in this article to aggregate neighboring node features. To balance local feature capturing ability and computational efficiency, we construct a GCN model architecture as shown in Fig. 3. It consists of two stacked graph convolutional layers with ReLU activation functions. This architecture aims to aggregate feature information from neighboring nodes. Employing two GCN layers allows us to effectively aggregate neighbor node information within two hops at a relatively low cost. Although increasing the number of layers may help expand the propagation range of neighbor node information, it can also introduce issues such as overfitting, increased computational costs, and gradient vanishing or exploding gradients.

Challenge 2: Relying solely on neighbor node features can diminish the overall role and impact of a node in a circuit, and it can lead to the repeated occurrence of specific local structural patterns throughout the circuit structures. This can

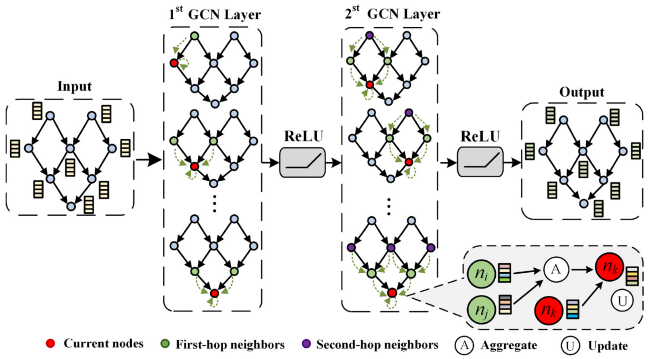

Fig. 3. GCN aggregating neighbor node features.

easily trap calculation models in local optima. Consequently, it is urgent to develop effective methods for extracting and utilizing global circuit feature information to enhance node information perception.

Solution 2: To address this challenge, we propose a circuit global feature information fusion method based on an attention mechanism. Our method first extracts an initial global feature vector for a circuit and iteratively updates it to obtain a global feature vector that effectively fuses information from all nodes. Subsequently, this global feature vector is combined with the node feature vector obtained after aggregating neighbor information, thus effectively enhancing the feature representation of circuit nodes.

This article proposes a method based on the attention mechanism (as shown in Fig. 4) to fuse global features. Initially, an average pooling operation of the feature vector matrix after GCN aggregation of neighboring node information is performed to obtain the initial global feature vector (denoted as α_0). Next, inner product operations are conducted between each node feature vector and α_0. The results are then normalized to derive a weight vector W, which reflects the contribution of each node to the initial global feature vector α_0. Subsequently, each node feature vector is weighted by the corresponding weight value and summed to obtain an updated global feature vector (denoted as α_1). The above operations are repeated to extract an effective global feature vector (α_2). This vector fuses the information of various node feature vectors, which helps capture the characteristic information of nodes in the graph while avoiding negative effects on generalization capabilities. Finally, α_2 is combined with each node's feature vector x_i through a weighted fusion operation to obtain the final node feature vector matrix. In fact, the extraction and fusion of local and global features inevitably involve overlapping or intersecting parts. To address this, after extracting the global features, an iterative update strategy for global features based on the attention mechanism is adopted. This allows the global feature vector and the node feature vector to update the global feature vector iteratively until it converges through the weighted summation of the node feature matrix using the weight coefficients derived from the inner product. This not only reduces the feature overlap caused by the fusion of local and global features but also highlights the contribution of each node's features to the global features, thereby reducing the adverse impact of residual overlap on the results. Details of the specific process can be found in Algorithm 2. The terms used are as follows: X as the initial feature matrix, A as the adjacency matrix, $\widetilde{A}$ as the symmetrically normalized Laplacian matrix of $\hat{A}$, $\hat{D}$ as the diagonal matrix, I_N as the unit matrix obtained by adding self-loop connections to A, $W^{(l)}$ as the weight matrix for the lth layer, σ as the activation function [4], *eb* as the node feature matrix after aggregating neighboring features, *neb* as the node feature matrix after aggregating global features, *ip* as the inner product value between the node feature vector and the global feature vector, α_c as the global feature vector derived from the cth iteration, *gf* as the final global feature vector after iterative update, x_i as the feature vector of node n_i, *avp* as the average pooling operation, *nml* as the normalization operation, and $\odot$ representing the element-wise multiplication operation.

In Algorithm 2, we commence by leveraging the graph convolution operation from [4] to aggregate feature information from both one-hop and two-hop neighboring nodes by utilizing the acquired node feature matrix X and adjacency matrix A. This operation yields an updated node feature matrix *eb*. Assuming that the time overhead of aggregating a single graph convolution layer is $O(Gc)$, the time overhead of fusing the local features of two graph convolution layers is $O(2 \times Gc)$. Next, steps 6 to 9 are employed to update α_0 twice, thereby effectively gathering information from all nodes in the graph to generate the final global feature vector *gf*. If the time overhead of a single iteration is $O(Ic)$, the time overhead of performing two iterative operations is $O(2 \times Ic)$. Subsequently, the vectors in node feature matrix *eb* are weighted and merged with the global feature vector *gf* to obtain the final node feature matrix *neb*. Considering that the node feature vector has a dimension of d, the time overhead of this process is $O(d \times N)$. In summary, since the primary time consumption of Algorithm 2 is attributed to the fusion of local features and the iterative update and fusion of the global feature vector, its overall time overhead is $O(2 \times Gc + 2 \times Ic + d \times N)$.

C. Imbalanced Dataset Handling

In IC designs, the small volume occupied by HTs and the minimal number of their gates constitute only a tiny fraction of the IC design. Consequently, there is an inevitable occurrence of severe dataset imbalance in the training dataset. Specifically, the number of normal gates is much larger than that of HT gates, which leads to the model being prone to overfitting the majority class during training and consequently results in poor recognition ability for the minority class.

Challenge 3: Due to the scarcity of HT nodes, traditional data imbalance handling methods that employ over-sampling or under-sampling strategies can introduce noise during training, which leads to information loss and potentially results in overfitting or underfitting issues. This can ultimately affect the generalization performance of the calculation models.

Solution 3: To address this challenge, we propose a data processing method that combines sampling with mini-batching during the construction of small-batch datasets. This approach simultaneously increases sample diversity and accelerates

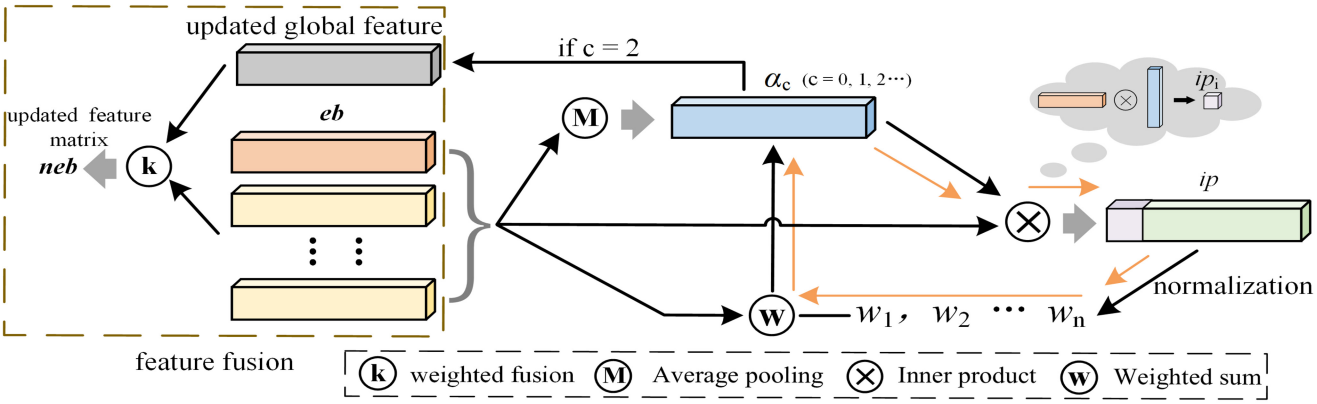

Fig. 4. Global feature extraction, update and fusion.

Algorithm 2 Feature Fusing

Require: X, A
Ensure: neb
1: $H(0) = X$;
2: $\hat{A} = A + I_N, \hat{D} = \sum_j \hat{A}_{ij}, \tilde{A} = \hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}$;
3: **Repeat** $H(l) \leftarrow \sigma(\tilde{A}H(l\text{-}1)W^{(l-1)})$ **When** $l = 1$ to 2;
4: $eb = H$;
5: $\alpha_0 = \text{avp}(eb)$;
6: **for** $c = 0$ to 1 **do**
7: **Repeat** $ip_i = eb_i \odot \alpha_c$ **when** $i = 1$ to N
8: $w = \text{nml}(ip)$ and $\alpha_{c+1} = \sum_{i=1}^{N} x_i \times w_i$;
9: **end for**
10: $gf = \alpha$ and $neb = k \times eb + (1 - k) \times gf$;
11: **return** neb.

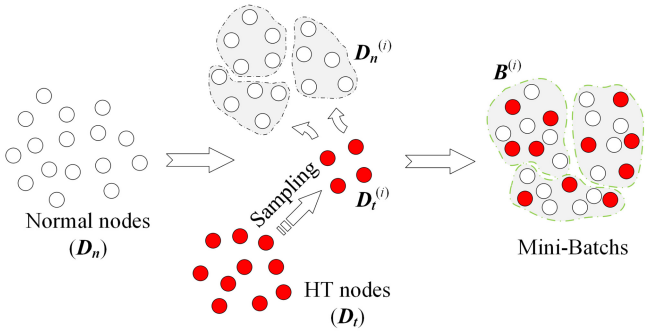

Fig. 5. Imbalanced dataset handling.

Algorithm 3 Mini-Batch Dataset Generating

Require: D_t, D_n, m
Ensure: B
1: $B \Leftarrow \emptyset$;
2: Split D_n into m subsets $D_n^{(i)}$ randomly, where $i \in [m]$;
3: **for** $i = 1$ to m **do**
4: $D_t^{(i)} \leftarrow \text{Sampling}(D_t)$ randomly, where $D_t^{(i)} \leq D_n^{(i)}$;
5: $B^{(i)} \leftarrow \{D_n^{(i)}\}$;
6: $B \leftarrow B \cup B^{(i)}$;
7: **end for**
8: **return** B

model training while also balancing the sample distribution and avoiding potential overfitting during training. Fig. 5 demonstrates a computation schematic of this method, with the detailed process explained in Algorithm 3. Here, D_t and D_n represent the sets of HT nodes and normal nodes, respectively, with their node counts being p and q, and m denotes the number of mini-batches, while B represents the generated mini-batch set.

In Algorithm 3, we commence by initializing mini-batch set $\boldsymbol{B}$. Next, we randomly divide normal node set $\boldsymbol{D_n}$ into m equally sized subsets $\boldsymbol{D_n^{(i)}}(i \in [m])$ based on $\boldsymbol{D_n}$ and mini-batch size m. Since it is necessary to traverse all nodes in the set, with the number of nodes being q, the time overhead is approximately $O(q)$. Afterward, we randomly sample m times from $\boldsymbol{D_t}$ and place the sampled nodes the subsets $\boldsymbol{D_t^{(i)}}(i \in [m])$ of HTs. It is required that the number of nodes in $\boldsymbol{D_t^{(i)}}$ approaches that of $\boldsymbol{D_n^{(i)}}$. Since the time cost of a single sampling is $O(q/m)$ and the entire process needs m iterations, the time overhead is approximately $O(q)$. Finally, we merge $\boldsymbol{D_n^{(i)}}$ and $\boldsymbol{D_t^{(i)}}$ to obtain mini-batch sets $\boldsymbol{B}$ $(i \in [m])$. Since the time cost for a single merge is $O(2q/m)$ and the entire computation requires m iterations, the time overhead of this step is approximately $O(2q)$. In summary, the primary time costs associated with this algorithm stem from partitioning the node set of size q, conducting m sampling iterations, and merging m subsets of size $2q/m$ each. Consequently, the overall time overhead of Algorithm 3 is approximately $O(4q)$.

IV. Experiment Setup

Threat Model: As per convention [4], in experiments, we presume that malicious attackers have successfully implanted HT triggers into normal circuits, and these triggers could be inserted into any position within a circuit, including the primary inputs, intermediate nodes, and the primary outputs. The threat model considered in the present paper covers a variety of possible attack paths, specifically HTs implanted at different design stages of ICs from untrusted 3PIP cores or unreliable electronic design automation (EDA) tools. We do not make special assumptions about HT triggers. Our proposed method can be used to detect HTs based on conditional

TABLE II
INFREQUENTLY USED CIRCUITS IN TRUST-HUB

Circuits	Normal nodes	HT nodes	Ratio of Nodes
EthernetMAC10GE-T700	102453	13	0.00013
EthernetMAC10GE-T710	102453	13	0.00013
EthernetMAC10GE-T720	102453	13	0.00013
EthernetMAC10GE-T730	102453	13	0.00013
B19-T100	63170	83	0.0013
B19-T200	63170	83	0.0013
wb-conmax-T100	23194	15	0.00065
Total	559346	233	-

triggering and always-on HTs [31]. Always-on HTs only contain payloads, while HTs with triggering mechanisms are composed of HT triggers and payloads. The trigger mechanisms can be combinational or sequential. The above two trigger types form the basis of almost all published trigger mechanisms for HTs.

Dataset: Our experiments utilize the widely accepted and utilized Trust-Hub and TRIT-TC benchmarks. Specifically, Trust-Hub benchmark includes 17 frequently used combinational and sequential circuits (such as RS232, S38417, S35932, and S38584) as well as 7 infrequently used circuits (such as wb-conmax-T100, B19, and EthernetMAC10GE). See Table II for details. Meanwhile, the TRIT-TC benchmark includes 15 combinational and sequential circuits, such as c2670, c3540, s1423, and s13207. For the detection of HTs in ICs at the design stage, these benchmarks provide circuits that cover various types and structures of HTs, which can be used to effectively evaluate the performance of HT detection and localization methods under various scenarios [2], [3], [4], [5]. In these benchmarks, HTs of different structures are embedded into various circuit types, and the insertion points are indicated by comments in the netlist code. All experiments are carried out on a 3.90GHz 16-core 64-bit Linux machine with 128GB memory and an NVIDIA GeForce RTX 3090 Ti.

Gate Modification Attacks: An adversarial dataset generated by gate modification attacks is often used to test the robustness of new methods under adversarial attacks [6]. The logical function of the affected gate nodes usually remains unchanged with this modification [6].

Evaluation Metrics: In this article, we adopt the widely-used leave-one-out cross-validation method [4], [6] to assess the performance of the constructed model. Five metrics commonly utilized to evaluate the performance of classification models are employed here to measure the HT detection capability of the constructed HTs-GCN: recall, precision (Pre), F1-score, accuracy(Acc), and TNR. HT nodes are viewed as positive samples. The specific calculation formulas can be seen in (5) and (6). TP represents correctly identified HT nodes, TN represents correctly identified normal nodes, FP represents normal nodes incorrectly identified as HT nodes, and FN represents HT nodes incorrectly identified as normal nodes

$$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}} \quad \text{Pre} = \frac{\text{TP}}{\text{TP}+\text{FP}} \quad F1\text{-score} = \frac{2\times\text{Pre}\times\text{Recall}}{\text{Pre}+\text{Recall}} \tag{5}$$

$$\text{Acc} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}} \quad \text{TNR} = \frac{\text{TN}}{\text{TN}+\text{FP}}. \tag{6}$$

V. EXPERIMENTAL RESULTS

A. Performance Analysis

To verify the performance of the proposed method, this article compares the HT detection performance of the proposed HTs-GCN method with other methods presented in the literature [2], [3], [4], [5]. We compare the methods on both frequently and infrequently used circuits in the Trust-Hub benchmark, and the results are shown in Table III. For ease of presentation, we give the average values of the five metrics provided in (5) and (6) for the five methods on the 17 frequently used circuits, 4 EthernetMAC10GE circuits, and 2 B19 circuits. To facilitate comparison, we also calculate the average values of the five metrics on all experimental circuits for the five methods. Note that "—" indicates that the corresponding method fails to give the corresponding data.

As can be seen from Table III, on the 17 frequently used circuits, although the proposed method is slightly weaker than NHTD-GL and the method in [3] in terms of the Pre, it has advantages in the other four metrics. Compared with the latest method NHTD-GL, under the circumstances where both Acc and TNR are the same and both perform excellently. Furthermore, the proposed method has significant advantages in recall and F1-score, which are about $\langle 10.2, 4.3\rangle$ percentage points better, respectively. Compared with the methods in [3] and [2], the proposed method is better in recall, Pre, F1-score, Acc, and TNR by about $\langle 41.7, -0.8, 36.2, 0.8, 0\rangle$ percentage points and $\langle 15.4, 7.4, 13.1, 1.6, 1.0\rangle$ percentage points on average, respectively. Although HTs-GCN is slightly weaker than TrojanSAINT [5] in terms of recall, it has a significant advantage in terms of TNR, which is about 3.7% points higher. On the infrequently used circuits EthernetMAC10GE, B19, and wb-conmax-T100, the proposed method still has advantages over NHTD-GL and the method in [3]: our method has an average advantage in recall, Pre, F1-score, Acc, and TNR of about $\langle 2.1, -0.9, 0.8, 0, 0\rangle$ percentage points and $\langle 12.5, -2.3, 10.5, 0, 0\rangle$ percentage points, respectively. Since the method in [2] and TrojanSAINT [5] fail to give any results for the above large circuits, we make no comparisons with them. The proposed method also successfully identifies all HTs on the large-scale circuits EthernetMAC10GE and B19. On all experimental circuits, the proposed method still has significant advantages over NHTD-GL, the method in [3], and the method in [2]: the average advantages in recall, Pre, F1-score, Acc, and TNR of about $\langle 7.8, -3.3, 3.3, 0, 0\rangle$ percentage points, $\langle 33.2, -1.2, 28.7, 0.4, 0\rangle$ percentage points, and $\langle 14.3, 7.9, 12.7, 1.5, 0.9\rangle$ percentage points, respectively. Compared with the TrojanSAINT [5], the average advantages in terms of recall and TNR are approximately $\langle -1.5, 3.7\rangle$ percentage points.

The proposed method quantifies the connectivity between nodes using Cov, and it obtains the Lfp of each node through approximate calculation, which strengthens the characteristics

TABLE III
COMPARISON WITH SIMILAR METHODS

Circuits	Method	Recall	Pre	F1-score	Acc	TNR
Frequently used circuits	HTs-GCN	0.967	0.931	0.946	0.997	0.999
	NHTD-GL [4]	0.865	0.974	0.903	0.997	0.999
	Method in [3]	0.550	0.939	0.584	0.989	0.999
	Method in [2]	0.813	0.857	0.815	0.981	0.989
	TrojanSAINT [5]	0.982	-	-	-	0.962
EthernetMAC10GE	HTs-GCN	1.000	0.982	0.991	1.000	1.000
	NHTD-GL [4]	0.981	1.000	0.990	1.000	1.000
	Method in [3]	0.958	1.000	0.977	1.000	1.000
	Method in [2]	-	-	-	-	-
	TrojanSAINT [5]	-	-	-	-	-
B19	HTs-GCN	1.000	0.994	0.997	1.000	1.000
	NHTD-GL [4]	1.000	0.951	0.975	1.000	1.000
	Method in [3]	1.000	1.000	1.000	1.000	1.000
	Method in [2]	-	-	-	-	-
	TrojanSAINT [5]	-	-	-	-	-
wb-conmax-T100	HTs-GCN	0.800	0.923	0.857	1.000	1.000
	NHTD-GL [4]	0.733	1.000	0.846	1.000	1.000
	Method in [3]	0.091	1.000	0.167	1.000	1.000
	Method in [2]	1.000	1.000	1.000	1.000	1.000
	TrojanSAINT [5]	-	-	-	-	-
Avg. on the above circuits	HTs-GCN	0.968	0.945	0.954	0.998	0.999
	NHTD-GL [4]	0.890	0.978	0.921	0.998	0.999
	Method in [3]	0.636	0.957	0.667	0.994	0.999
	Method in [2]	0.825	0.866	0.827	0.983	0.990
	TrojanSAINT [5]	0.982	-	-	-	0.962

of high connectivity and low logical flipping of HT nodes. Moreover, our proposed method enhances the information perception ability of nodes by fusing nodes' local and global features. Application of a dataset imbalance handling method further improves the ability of the proposed method to solve the classification deviation problem. However, the new features Cov and Lfp of the HTs-GCN method are likely to misclassify a few normal nodes with high connectivity or low Lfp as HT nodes with the same features. Although NHTD-GL also considers the difference in the Lfp of different nodes in the graph, it fails to accurately identify HT nodes with high impact or in critical locations because it ignores the connectivity information between nodes in a graph and does not fuse the global information of nodes. The TrojanSAINT [5] method, which is based on generating subgraphs through sampling, can easily meet the sample requirements of small-scale circuits. However, in large-scale circuits, the increased number of node neighbors often significantly amplifies the sample demand for generating effective subgraphs. This makes it difficult for rapid sampling to effectively cover node neighbor information, thus leading to computational bias issues. The method in [2] uses ADASYN to generate new minority class samples to handle an imbalanced dataset, which increases the possibility of introducing noise. The method in [3] ignores the handling of dataset imbalance issues, which affects the classification performance of the bagged tree model. We also found that the method in [2] ignores the difference in functional features of different nodes; meanwhile, the method in [3] ignores the difference in structural features of different nodes in a graph. In addition, the lack of feature fusion in TrojanSAINT [5] and methods presented in [2] and [3] prevents them from fully exploiting and utilizing the useful information contained within nodes.

TABLE IV
EFFECT OF HTS-GCN ON IDENTIFYING HTS BEFORE AND AFTER APPLYING THE DIFFERENT MODULES

Method	Recall	Pre	F1-score	Acc
HTs-GCN	0.968	0.945	0.954	0.998
Sc.1	0.958	0.907	0.929	0.996
Sc.2	0.942	0.893	0.910	0.996
Sc.3	0.936	0.887	0.908	0.995
Sc.4	0.917	0.835	0.869	0.993
Sc.5	0.948	0.885	0.913	0.995

B. *Efficiency Investigation*

To validate the effectiveness of the proposed method, first, we compared the performance changes in HTs-GCN detection of HTs before and after adding new features Cov and Lfp. Subsequently, we assessed the difference in HTs-GCN's performance in detecting HTs, before and after implementing local and global feature fusion at the node level. Finally, we also looked at changes in the performance of HTs-GCN in detecting HTs, before and after applying the imbalanced dataset treatment method described in Section III-C. The experimental results are presented in Table IV, wherein Sc.1, Sc.2, Sc.3, Sc.4, and Sc.5, respectively, denote the scenarios where HTs-GCN does not add new feature Cov, does not add new feature Lfp, does not add new features Cov and Lfp, does not implement local and global feature fusion at the node level, and does not apply imbalanced dataset handling method.

As shown in Table IV, compared with HTs-GCN, Sc.1, Sc.2, Sc.3, Sc.4, and Sc.5, all experienced varying degrees of decline in four different evaluation metrics. However, relatively good recall was observed across all scenarios, with only approximately 1.0, 2.6, 3.2, 5.1, and 2.0% points losses, respectively. Among the other three metrics, the largest drops for Sc.1, Sc.2, Sc.3, Sc.4, and Sc.5 were observed in Pre,

TABLE V
DETECTION RESULTS ON INFREQUENTLY USED CIRCUITS

Netlist	Recall	Pre	F1-score	Acc
EthernetMAC10GE-T700	0.923	0.857	0.889	1.000
EthernetMAC10GE-T710	0.923	0.857	0.889	1.000
EthernetMAC10GE-T720	1.000	0.929	0.963	1.000
EthernetMAC10GE-T730	0.923	0.857	0.889	1.000
B19-T100	0.964	1.000	0.982	1.000
B19-T200	0.976	0.976	0.976	1.000
wb-conmax-T100	0.800	0.857	0.828	1.000
AES-T100	0.963	0.949	0.956	0.997
AES-T200	0.961	0.942	0.951	0.997
AES-T300	0.948	0.957	0.953	0.995
Average	0.938	0.918	0.927	0.999

where they declined by 3.8, 5.2, 5.8, 11.0, and 6.0% points, respectively, compared with HTs-GCN. These scenarios also showed weaker F1-scores, with decreases of approximately 2.5, 4.4, 4.6, 8.5, and 4.1% points, respectively. These five scenarios demonstrated the smallest declines in Acc, with a respective decline of only about 0.2, 0.2, 0.3, 0.5, and 0.3% points from HTs-GCN. Moreover, among the five schemes, Sc.1 yielded the best detection results, followed by Sc.5, Sc.2, and Sc.3; Sc.4 resulted in the worst performance.

The two new features of Cov and Lfp proposed in Section III-A, the fusion strategy of local and global features of nodes given in Section III-B, and the method of imbalanced dataset handling proposed in Section III-C not only enhance the feature expression ability and information perception ability of the nodes in a circuit but also enrich the feature information of the nodes in the circuit, and improve the ability of the model to handle classification bias. In fact, removing Cov and Lfp, or not merging the local and global features of nodes, will affect the feature expression ability of the nodes and reduce the model's ability to discern normal nodes. Compared with Cov, the removal of Lfp has a greater impact on the performance of HTs-GCN. Although the imbalanced dataset in the test set tends to cause the model to be biased toward predicting the majority class when trying to predict the minority class, well-trained models can usually correctly classify most normal nodes. Compared with Cov, the addition of Lfp is more effective in identifying HTs in circuits that commonly exhibit high concealment. Moreover, compared with the features Cov and Lfp and the handling of an imbalanced dataset, the merged features can often more effectively highlight the differences between HT nodes and normal nodes.

C. Application Analysis

To further evaluate the generalizability of the proposed method, we tested the performance of the HTs-GCN model trained on 17 frequently used circuits in the Trust-Hub benchmark to identify HTs in the seven infrequently used circuits (see Table II) and three AES circuits. The type of HTs in the AES circuits is always-on. The results are shown in Table V. It should be noted that other similar methods [2], [3], [4], [5] have not yet given similar experimental results, and therefore it is not possible to compare them with our method in terms of generalizability.

As can be seen from Table V, the proposed method performs similarly across the four EthernetMAC10GE circuits; it also has similar performance across the two B19 circuits and three AES circuits. Our method not only performs well in terms of Acc and recall (with average values of approximately 99.9% and 93.8%, respectively) but also has very good performance in terms of Pre and F1-score (with average values of approximately 91.8% and 92.7%, respectively). The proposed method accurately identifies the vast majority of normal nodes and HT nodes in the circuits. It performs best on B19, followed by EthernetMAC10GE, AES, and wb-conmax-T100. The four EthernetMAC10GE circuits have the same number of nodes, the same number of HT nodes, and roughly the same internal structure. The two B19 circuits and three AES circuits also have similar characteristics. Between different types of circuits, the performance of an HT detection method often varies due to differences in the number of nodes, internal structure, and so on. For AES circuits with always-on HTs, the proposed method also demonstrates excellent performance with an average recall of 95.7%. This is primarily attributed to the fact that always-on HTs, which ensure their destructive potential, often exhibit structural and functional characteristics that deviate from normal nodes. These distinctive features can be effectively captured by HTs-GCN. HTs-GCN's slightly weaker performance on wb-conmax-T100 may be due to its unique structure: for example, its HT triggers are mainly composed of XOR gates. In summary, compared with existing methods, the proposed HTs-GCN method has excellent performance in terms of key metrics and generalizability on different types and sizes of circuits, which lays an important foundation for its practical applications.

In addition, we tested the performance and robustness of HTs-GCN on the TRIT-TC benchmark. First, we compared its performance with that of R-HTDetector [6] when under non-adversarial attacks. Then, under gate modification attacks [6], we compared these methods' robustness. The results are shown in Table VI. For a fair comparison, we use the two evaluation indicators TPR and TNR given in the original paper on R-HTDetector [6], where TPR is also known as recall.

As can be seen from Table VI, when the circuits are not under attack, the performance of HTs-GCN is excellent and superior to that of R-HTDetector [6]. Specifically, HTs-GCN's average TPR and TNR are 95.1% and 94.4%, respectively; these which are about 0.2% points and 9.1% points higher than those of R-HTDetector [6], respectively. Under gate modification attacks, the average TPR and TNR of HTs-GCN are 82.1% and 92.5%, respectively, which are about 2.3% points and 7.1% points higher than those of R-HTDetector [6], respectively. The circuit attack causes HTs-GCN's TPR and TNR to decrease by 13.0% and 1.9% points, respectively. Meanwhile, R-HTDetector [6] experiences decreases of 15.1% and −0.1% points, respectively; despite this, R-HTDetector remains slightly less effective than HTs-GCN. HTs-GCN captures the features of a node itself, its neighbors, and global features, and then it uses the powerful feature learning ability of its GCN to identify HTs. This makes HTs-GCN applicable to various types of benchmarks. Gate modification attacks modify the logic gates within HTs, which often cause changes

TABLE VI
COMPARISON OF DETECTION PERFORMANCE BETWEEN HTS-GCN AND R-HTDETECTOR (TRIT-TC BENCHMARK)

Circuit names	Original circuits				Gate modification-attacked circuits			
	TPR		TNR		TPR		TNR	
	R-HTDetector [6]	HTs-GCN	R-HTDetector [6]	HTs-GCN	R-HTDetector [6]	HTs-GCN	R-HTDetector [6]	HTs-GCN
c2670-T000	1.000	1.000	0.859	0.928	0.800	0.750	0.863	0.922
c2670-T001	1.000	1.000	0.840	0.942	0.833	0.667	0.840	0.939
c2670-T002	0.750	1.000	0.909	0.932	0.625	0.750	0.909	0.932
c3540-T000	1.000	0.800	0.935	0.952	0.800	0.800	0.934	0.951
c3540-T001	1.000	1.000	0.646	0.934	0.833	0.833	0.646	0.933
c3540-T002	1.000	1.000	0.680	0.953	0.385	0.833	0.683	0.953
c5315-T000	0.875	0.875	0.784	0.965	0.750	0.750	0.784	0.964
c5315-T001	0.778	0.889	0.863	0.969	0.750	0.667	0.864	0.914
c5315-T002	1.000	1.000	0.710	0.964	0.800	0.800	0.712	0.917
s1423-T000	1.000	1.000	0.908	0.904	0.833	1.000	0.910	0.910
s1423-T001	0.833	0.833	0.919	0.901	0.929	0.833	0.921	0.901
s1423-T002	1.000	0.875	0.869	0.913	1.000	0.833	0.869	0.926
s13207-T000	1.000	1.000	0.962	0.970	0.769	0.800	0.962	0.904
s13207-T001	1.000	1.000	0.961	0.969	0.857	1.000	0.961	0.897
s13207-T002	1.000	1.000	0.955	0.968	1.000	1.000	0.955	0.917
Average	0.949	0.951	0.853	0.944	0.798	0.821	0.854	0.925

in node features, such as the degree and connectivity of nodes, and this leads to the model easily misjudging the nature of nodes. However, this type of attack still maintains the original logical function of a circuit, and therefore the functional features of the nodes are usually not greatly affected. R-HTDetector [6] mainly focuses on structural feature analysis while HTs-GCN integrates the structural and functional features of the circuits, which renders it relatively less affected by circuit structural changes.

VI. CONCLUSION

This article introduces an HTs-GCN method for the detection and localization of HTs in circuits. The proposed method constructs Cov and Lfp through a depth-first search strategy and a topological logic analysis method to enrich the feature information of circuit nodes. It then aggregates the local feature information of nodes and the global feature information of the circuits through a message-passing mechanism to update the feature vectors of nodes, thereby enhancing the expressiveness of the circuit node features. Then, based on the concept of stochastic gradient descent and incorporating mini-batch oversampling and under-sampling methods, the training data is balanced to accelerate model training and enhance its performance. Experimental results on the Trust-Hub and TRIT-TC benchmarks, which are highly recognized and widely used in the industry, verify the superior performance and excellent generalizability of the proposed method compared with other methods. In addition, the proposed method also exhibits a high degree of robustness against gate modification attacks.

REFERENCES

[1] K. I. Gubbi et al., "Hardware Trojan detection using machine learning: A tutorial," *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 3, pp. 1–26, Apr. 2023.

[2] C. H. Kok, C. Y. Ooi, M. Moghbel, N. Ismail, H. S. Choo, and M. Inoue, "Classification of Trojan nets based on SCOAP values using supervised learning," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2019, pp. 1–5.

[3] T. Kurihara and N. Togawa, "Hardware-Trojan classification based on the structure of trigger circuits utilizing random forests," in *Proc. IEEE 27th Int. Symp. On-Line Testing Robust System Design (IOLTS)*, Jun. 2021, pp. 1–4.

[4] K. Hasegawa, K. Yamashita, S. Hidano, K. Fukushima, K. Hashimoto, and N. Togawa, "Node-wise hardware Trojan detection based on graph learning," *IEEE Trans. Comput.*, early access, May 25, 2023, doi: 10.1109/TC.2023.3280134s.

[5] H. Lashen, L. Alrahis, J. Knechtel, and O. Sinanoglu, "TrojanSAINT: Gate-level Netlist sampling-based inductive learning for hardware Trojan detection," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Jul. 2023, pp. 1–5.

[6] K. Hasegawa, S. Hidano, K. Nozawa, S. Kiyomoto, and N. Togawa, "R-HTDetector: Robust hardware-Trojan detection based on adversarial training," *IEEE Trans. Comput.*, vol. 72, no. 2, pp. 333–345, Feb. 2023.

[7] V. Gohil, H. Guo, S. Patnaik, and J. Rajendran, "ATTRITION: Attacking static hardware Trojan detection techniques using reinforcement learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Nov. 2022, pp. 1275–1289.

[8] A. Bhattacharyay, S. Yang, J. Cruz, P. Chakraborty, S. Bhunia, and T. Hoque, "An automated framework for board-level Trojan benchmarking," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 397–410, Feb. 2023.

[9] K. Huang and Y. He, "Trigger identification using difference-amplified controllability and dynamic transition probability for hardware Trojan detection," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3387–3400, 2020.

[10] Z. Pan and P. Mishra, "TD-Zero: Automatic golden-free hardware Trojan detection using zero-shot learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 7, pp. 1998–2011, Jul. 2024.

[11] R. Yasaei, L. Chen, S.-Y. Yu, and M. A. Al Faruque, "Hardware Trojan detection using graph neural networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 1, pp. 25–38, Jan. 2025.

[12] Z. Pan and P. Mishra, "Design of AI Trojans for evading machine learning-based detection of hardware Trojans," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, Mar. 2022, pp. 682–687.

[13] R. Hassan, X. Meng, K. Basu, and S. M. P. Dinakarrao, "Circuit topology-aware vaccination-based hardware Trojan detection," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 9, pp. 2852–2862, Sep. 2023.

[14] S. Ghandali, T. Moos, A. Moradi, and C. Paar, "Side-channel hardware Trojan for provably-secure SCA-protected implementations," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 6, pp. 1435–1448, Jun. 2020.

[15] F. Zhang, Z. Wang, H. Shen, B. Yang, Q. Wu, and K. Ren, "DARPT: Defense against remote physical attack based on TDC in multi-tenant scenario," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, New York, NY, USA, 2022, pp. 559–564.

[16] J. Xiao, Z. Wu, and J. Lou, "FS-TRA: Evaluating sequential circuit reliability via a fanout-source tracking and reduction approach," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 12, pp. 4726–4739, Dec. 2024, doi: 10.1109/TCAD.2024.3410157.

[17] R. Yasaei, S. Faezi, and M. A. Al Faruque, "Golden reference-free hardware Trojan Localization using graph convolutional network," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 10, pp. 1401–1411, Oct. 2022.

[18] R. Yasaei, S.-Y. Yu, and M. A. Al Faruque, "GNN4TJ: Graph neural networks for hardware Trojan detection at register transfer level," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, Feb. 2021, pp. 1504–1509.
[19] S.-Y. Yu, R. Yasaei, Q. Zhou, T. Nguyen, and M. A. Al Faruque, "HW2VEC: A graph learning tool for automating hardware security," in *Proc. IEEE Int. Symp. Hardw. Orient. Security Trust (HOST)*, Dec. 2021, pp. 13–23.
[20] V. Gohil, S. Patnaik, H. Guo, D. Kalathil, and J. J. Rajendran, "DETERRENT: Detecting Trojans using reinforcement learning," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, New York, NY, USA, 2022, pp. 697–702, doi: 10.1145/3489517.3530518.
[21] P. Krishnamurthy, V. R. Surabhi, H. Pearce, R. Karri, and F. Khorrami, "Multi-modal side channel data driven golden-free detection of software and firmware Trojans," *IEEE Trans. Depend. Secure Comput.*, vol. 20, no. 6, pp. 4664–4677, Nov./Dec. 2023.
[22] A. Mondal, S. Karmakar, M. H. Mahalat, S. Roy, B. Sen, and A. Chattopadhyay, "Hardware Trojan detection using transition probability with minimal test vectors," *ACM Trans. Embed. Comput. Syst.*, vol. 22, no. 1, pp. 1–21, Oct. 2022.
[23] J. Xiao, W. Chen, J. Lou, J. Jiang, and Q. Zhou, "Identifying reliability-critical primary inputs of combinational circuits based on the model of gate-sensitive attributes," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4708–4720, Nov. 2022.
[24] A. Takiddin, R. Atat, M. Ismail, O. Boyaci, K. R. Davis, and E. Serpedin, "Generalized graph neural network-based detection of false data injection attacks in smart grids," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 3, pp. 618–630, Jun. 2023.
[25] D. Utyamishev and I. Partin-Vaisband, "Knowledge graph embedding and Visualization for pre-silicon detection of hardware Trojans," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2022, pp. 180–184.
[26] W. Pan et al., "A unioned graph neural network based hardware Trojan node detection," *IEICE Electron. Exp.*, vol. 20, no. 13, 2023, Art. no. 20230204.
[27] Z. Shi, H. Ma, Q. Zhang, Y. Liu, Y. Zhao, and J. He, "Test generation for hardware Trojan detection using correlation analysis and genetic algorithm," *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 4, pp. 1–20, Mar. 2021.
[28] K. Khatua, "Low power state assignment of sequential circuits based on binary particle swarm optimization and flip-flop selection," in *Proc. Int. Conf. Elect., Comput. Energy Technol. (ICECET)*, Jul. 2022, pp. 1–6.
[29] J. Xiao, W. Zhu, Q. Shen, H. Long, and J. Lou, "A pruning and feedback strategy for locating reliability-critical gates in combinational circuits," *IEEE Trans. Rel.*, vol. 72, no. 3, pp. 1078–1092, Sep. 2023.
[30] J. Xiao, Y. Yang, H. Long, R. Qin, and J. Lou, "Estimating redundancy-reliability of CNNs based on strip-median attributes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 10, pp. 1486–1496, Oct. 2023.
[31] Y. Yang et al., "Survey: Hardware Trojan detection for Netlist," in *Proc. IEEE 29th Asian Test Symp. (ATS)*, 2020, pp. 1–6.

Jie Xiao (Member, IEEE) received the Ph.D. degree in computer system architecture from Tongji University, Shanghai, China, in 2013.

He is currently with the Department of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China. He has published more than 60 papers in refereed journals, including IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, IEEE TRANSACTIONS ON RELIABILITY, and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS. His current research interests include reliability calculation and fault-tolerant design.

Shuiliang Chai (Graduate Student Member, IEEE) received the bachelor's degree in the Department of Data Science and Big Data Technology, Zhejiang University of Finance and Economics, Hangzhou, China, in 2022. He is currently pursuing the master's degree with the School of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China.

His current research interests include hardware Trojan detection and circuit vulnerability assessment.

Yanjiao Gao is currently pursuing the master's degree with the School of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China.

Her current research interests include AI security testing and assessment, machine learning, and big data mining.

Yuhao Huang is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China.

His research interests include AI security and reliability, deep learning, and big data mining.

Fan Zhang (Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, Storrs, CT, USA, in 2012.

He is currently a Full Professor with the School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, Hangzhou, China, where he is also with the College of Information Science and Electronic Engineering. His research interests include hardware security, system security, and computer architecture.

Tieming Chen received the Ph.D. degree in computer software and theory from Beihang University, Beijing, China, in 2011.

He is currently a Professor with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China. His research interests include cyberspace security and data intelligence security.

Prof. Chen is also a member of ACM and CCF.